Nimo Al-Azzawi

Term Project: Instant Messaging App

Feature Document

Table of Contents

1 Introduction 2

1.1 Purpose 2

1.2 Target Audience 2

1.3 Terms and Definitions 2

2 Product Overview 4

2.1 Product Vision 4

2.2 Product Scope 4

2.3 Operating Environment 5

2.4 Design and Implementation Constraints 5

3 User Stories 7

3.1 Personas 7

3.2 User Scenarios 8

3.2.1 <User Scenarios with Persona 1> 8

3.2.2 <User Scenarios with Persona 2> 8

3.2.3 <User Scenarios with Persona 3> 9

3.3 Epic User Stories 9

3.3.1 <Epic User Stories 1> 9

3.3.2 <Epic User Stories 2> 10

3.3.3 <Epic User Stories 3> 11

4 Feature List 13

1 Introduction

This document describes the features, scope, and design considerations for an Instant Messaging Application similar to Yahoo Massanger or MSN. It serves as a planning and reference document throughout development, ensuring that user needs, system requirements, and implementation constraints are clearly defined before and during development.

1.1 Purpose

The purpose of this document is to define the scope, features, and user experience of the frontend portion of the instant messaging application. It serves as a guide for implementing the user interface and integrating with a backend system supplied by the TA. This document also helps demonstrate clear separation of frontend and backend responsibilities.

1.2 Target Audience

- This document is intended for the development team (Who is working on frontend), the course instructor, teaching assistant, and any future developers who may work on or maintain this application. It provides the necessary context and requirements for anyone involved in the project.

1.3 Terms and Definitions

- Chat Room: A virtual space where multiple users can exchange messages in real-time
- Client: The user-facing application (frontend) that connects to the server
- Server: The backend application that handles client connections, message routing, and data persistence
- Real-time Messaging: Messages delivered with minimal latency, appearing instantly to recipients

- Authentication: The process of verifying user identity through registration and login
- MERN Stack: MongoDB, Express, React, Node.js - the technology stack used for development
- API: Application Programming Interface - defines how frontend and backend communicate.

2 Product Overview

This product is a web-based instant messaging client designed for users who want simple, nostalgic real-time chat. It targets students and casual users who want a simple, nostalgic chat experience inspired by early instant messaging applications.
WHAT: A frontend instant messaging application
WHO: Students and casual users
WHY: To provide simple, organized, real-time communication with a classic UI

2.1 Product Vision

For students and casual users Who want simple and familiar instant messaging The Instant Messaging Application Is a web-based chat client That provides real-time chat rooms with a Yahoo Messenger / MSN-style interface Unlike modern messaging apps with complex interfaces Our product focuses on simplicity, clarity, and nostalgic design

2.2 Product Scope

Instant Messaging App (Frontend)

- Authentication UI
    - Login
    - Registration
    - Logout
- User Profile
    - Set first name
    - Set last name
- Chat Rooms
    - List chat rooms (sidebar layout)
    - Create chat room
    - Delete chat room
    - Navigate between chat rooms
- Messaging

    - Message input box
    - Message display area
    - Message timestamps
    - Auto-scroll on new messages
    - Send text messages in real-time
    - Receive messages instantly
    - View conversation history
- Visual Design
    - MSN/Yahoo-inspired layout
    - Classic color palette
    - Status indicators
- Data Persistence
    - Store user credentials securely
    - Maintain message history
    - Track chat room membership
    - Persist user profile information
- Error Handling
    - Authentication errors
    - Connection errors

2.3 Operating Environment

- Client-Side: Modern web browsers (Chrome, Firefox, Safari, Edge) with JavaScript enabled
- Server-Side: Node.js
- Database: MongoDB database system
- Network: Stable internet connection for real-time messaging
- Devices: Desktop computers, laptops, tablets with responsive screen support.

2.4 Design and Implementation Constraints

Technology Stack:

- Frontend: React.js for building the user interface
- Backend: Node.js with Express.js framework (if a backend needed to be build)
- Database: MongoDB for data storage
- Version Control: Git (GitHub/Bitbucket) for source code management.

Constraints:

- Individual developers must choose either frontend or backend focus
- Real-time messaging requires WebSocket or similar protocol implementation
- Security requirements for password storage and session management
- Project timeline constrained to 11-week term
- Must maintain compatibility with TA-provided complementary component
- Browser compatibility requirements limit certain modern features

3 User Stories

Understanding users' needs is essential for designing an intuitive frontend interface. User stories help translate user expectations into UI behaviors and interactions without involving backend implementation details.

3.1 Personas

Persona 1: Sarah - The College Student

- Age: 20
- Occupation: Computer Science major
- Tech Savvy: High - comfortable with multiple apps and platforms
- Goals: Coordinate group projects, quick questions to classmates, share study resources
- Pain Points: Email too slow, SMS group chats get messy, wants organized topic-based conversations
- Quote: "I need to keep my class discussions separate from my personal chats."

Persona 2: Marcus - The Remote Team Lead

- Age: 32
- Occupation: Software engineering team lead
- Tech Savvy: Very high - works with technical tools daily
- Goals: Quick team coordination, manage multiple project channels, reference past decisions
- Pain Points: Enterprise tools too complex for small team, needs simple yet nostalgic deployment, values message history
- Quote: "I need something lightweight that my whole team can use without a learning curve."

Persona 3: Elena - The Community Organizer

- Age: 45
- Occupation: Volunteer coordinator for local non-profit

- Tech Savvy: Moderate - uses email and social media regularly
- Goals: Coordinate volunteers, share event updates, maintain committee discussions
- Pain Points: Facebook groups too public, email chains confusing, needs affordable solution
- Quote: "I want everyone to stay informed without overwhelming them with notifications."

3.2 User Scenarios

3.2.1 <User Scenarios with Sarah (College Student)>

Scenario 1: Group Project Coordination

Sarah has a software engineering group project with four classmates. She creates a new chat room called “SE Project Team” and shares it with her teammates. Throughout the week, the team uses this room to share code snippets, ask quick questions, and coordinate meeting times. Sarah appreciates being able to scroll back through the conversation history to find the database schema they discussed earlier.

Scenario 2: Study Group Management

Before midterms, Sarah creates multiple chat rooms: one for her data structures study group, another for her algorithms class, and a third for general CS questions. She easily switches between rooms throughout the day, keeping her academic discussions organized by topic rather than having everything mixed in one long thread.

3.2.2 <User Scenarios with Marcus (Remote Team Lead)>

Scenario 1: Sprint Planning

Marcus sets up chat rooms for different projects his team is working on. During sprint planning, he creates a “Sprint 5 Discussion” room where the team discusses task assignments in real-time. He can reference messages from the previous sprint's room to review what worked well and what needs improvement.

Scenario 2: Quick Technical Decisions

When a critical bug appears in production, Marcus quickly sends a message to the "Backend Team" chat room. Within minutes, three developers respond with their analysis. The real-time nature of the messaging allows them to resolve the issue much faster than email exchanges would have allowed.

3.2.3 <User Scenarios with Elena (Community Organizer)>

Scenario 1: Event Planning

Elena creates a chat room for each upcoming community event. For the annual fundraiser, she sets up "Fundraiser 2026" and invites all committee members. As volunteers join the room, they can immediately see past decisions about venue, catering, and logistics without Elena having to repeat herself.

Scenario 2: Volunteer Coordination

When last-minute changes occur, Elena posts updates to the relevant chat room knowing all volunteers will see the information the next time they check. She doesn't worry about messages getting lost in email spam folders or social media feeds. The persistent message history ensures late joiners can catch up on what they missed.

3.3 Epic User Stories

Write at least three epic (Chapter 3 p28 3rd bullet point) user stories (Chapter 3 p27-29) with acceptance criteria in the following template. Try to define each small user story specifically so that they could directly be used in the sprint backlog.

3.3.1 < Epic 1: User Authentication and Profile Management>

Epic 1: User Authentication and Profile Management
As a new user ***I want to register an account, securely log in and out, and customize my profile with my name*** ***So that I can establish my identity within the application and access personalized messaging features***
Acceptance Criteria **Given I am a new user accessing the application** **When I complete registration with valid credentials and log in** **Then I should gain authenticated access to the messaging system, be able to set my first and last name, and securely log out when finished**
<Breakdown the features to a single feature that can be implemented in a sprint> User Story 1.1: User Registration As a new user, I want to create an account with a unique username and password so that I can access the messaging application.
User Story 1.2: User Login As a registered user, I want to log in with my credentials so that I can access my chat rooms and messages.
User Story 1.3: Profile Customization As a user, I want to set and update my first and last name so that other users can identify me properly in conversations.

3.3.2 < Epic 2: Chat Room Management>

Epic 2: Chat Room Management

As a user ***I want to create, view, join, navigate between, and delete chat rooms*** ***So that I can organize my conversations by topic, project, or group and maintain multiple discussion spaces***
Acceptance Criteria **Given I am a logged-in user** **When I create a new chat room or select an existing one from the list** **Then I should be able to navigate to that room, see its message history, and delete rooms I no longer need while seeing all available rooms in an organized list**
<Breakdown the features to a single feature that can be implemented in a sprint> User Story 2.1: Create Chat Room As a user, I want to create a new chat room with a custom name so that I can start a focused conversation on a specific topic.
User Story 2.2: View Chat Room List As a user, I want to see a list of all available chat rooms so that I can choose which conversations to join.
User Story 2.3: Navigate Between Rooms As a user, I want to easily switch between different chat rooms so that I can participate in multiple conversations without confusion.

3.3.3 <Real-Time Messaging and History>

Epic 2: Chat Room Management

As a chat room participant ***I want to send and receive messages instantly while being able to view the complete conversation history*** ***So that I can communicate effectively with other users in real-time and reference past discussions when needed***
Acceptance Criteria **Given I am in an active chat room with other participants** **When I send a message or another user sends a message** **Then the message should appear immediately for all participants, display the sender's name and timestamp, and all previous messages should remain accessible through conversation history**
<Breakdown the features to a single feature that can be implemented in a sprint> User Story 3.1: Send Messages As a user in a chat room, I want to type and send text messages so that I can communicate with other participants.
User Story 3.2: Receive Messages in Real-Time As a user in a chat room, I want to see new messages appear instantly without refreshing so that I can follow the conversation as it happens.
User Story 3.3: View Conversation History As a user, I want to scroll through past messages in a chat room so that I can review earlier discussions and context.

4 Feature List

The following comprehensive feature list prioritizes items based on their importance to core functionality and estimates implementation effort. Features are sorted with high-priority, low-effort items first to maximize value delivery in early sprints. This list serves as the primary reference for sprint planning and backlog creation.

Feature 1: Authentication UI	**Priority:High**	**Effort Required:** (2)
Description: : Login, registration, and logout screens		
Constraints: Backend logic handled by TA		
Comments: Frontend-only responsibility		

Feature 2: Chat Room Sidebar	**Priority:High**	**Effort Required:** (2)
Description: Sidebar-based room list and switching		
Constraints: Depends on backend APIs		
Comments: Core navigation		

Feature 3: Messaging Interface	**Priority:High**	**Effort Required:** (4)

Description: Yahoo/MSN-style chat window
Constraints: Real-time data provided by backend
Comments: Primary user interaction

Feature 4: Visual Styling	**Priority: Mid**	**Effort Required:** (2)
Description: Display backend and connection errors		
Constraints: Frontend detection only		
Comments: Improves clarity		

Feature 5: Visual Styling	**Priority: low**	**Effort Required:** (1)
Description: Nostalgic theme and layout		
Constraints: Browser compatibility		
Comments: Enhances usability		